SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta informatiky a informačných technológií

Evidenčné číslo: FIIT-16768-120896

Generátor plošinových hier

Bakalárska práca

Študijný program: informatika
Študijný odbor: informatika
Školiace pracovisko: Ústav informatiky, informačných systémov a softvérového inžinierstva
Vedúci záverečnej práce: Mgr. Matej Pecháč, PhD.

Bratislava 2025 **Branislav Trstenský**

Slovenská technická univerzita v Bratislave
Ústav informatiky, informačných systémov a softvérového inžinierstva

Fakulta informatiky a informačných technológií
2024/2025

ZADANIE BAKALÁRSKEJ PRÁCE

Autor práce:	Branislav Trstenský
Študijný program:	informatika
Študijný odbor:	informatika
Evidenčné číslo:	FIIT-16768-120896
ID študenta:	120896
Vedúci práce:	Mgr. Matej Pecháč, PhD.
Vedúci pracoviska:	doc. Ing. Ján Lang, PhD.
Názov práce:	**Generátor plošinových hier**
Jazyk, v ktorom sa práca vypracuje:	slovenský jazyk
Špecifikácia zadania:	V oblasti učenia posilňovaním - oblasť umelej inteligencie, ktorá sa zaoberá rozhodovaním agentov - sa používajú na tréning a hodnotenie staré ATARI hry. Tieto hry bežia v emulátore, co značne spomaľuje tréning. Zároveň nemáme plný prístup k ich stavu, co nám neumožňuje analyzovať agentov na celom stavovom priestore. Analyzujte existujúce ATARI hry Montezuma's Revenge a Pitfall!, ktoré patria medzi najzložitejšie a zároveň najzaujímavejšie z pohľadu výskumu. Navrhnite generátor, ktorý by dokázal vytvárať úrovne podobné týmto hrám a skopíroval ich mechaniky. Užívateľ by si mal vedieť sám generovať úrovne na základe niekoľkých parametrov, ktorými nastavuje ich zložitosť. Implementujte takýto generátor spolu s mechanikami a grafickým rozhraním v C++ alebo Python (odporúča sa). Grafika bude dostupná v rámci zadania. Sústreďte sa na to, aby efektívne a rýchlo bežalo paralelne viacero inštancií hry (napr. 128). Na záver spravte manuál, ako generátor používať a pripravte niekoľko ukážkových úrovní.
Rozsah práce:	40
Termín odovzdania práce:	12. 05. 2025

Čestne vyhlasujem, že som túto prácu vypracoval(a) samostatne, na základe konzultácií a s použitím uvedenej literatúry.

V Bratislave, 21. februára 2025.

Branislav Trstenský

Anotácia

Slovenská technická univerzita v Bratislave

FAKULTA INFORMATIKY A INFORMAČNÝCH TECHNOLÓGIÍ

Študijný program:	informatika
Autor:	Branislav Trstenský
Bakalárska práca:	Generátor plošinových hier
Vedúci diplomovej práce:	Mgr. Matej Pecháč, PhD.

Január 2025

Pri spätnoväzobovom učení, v oblasti umelej inteligencie zaoberajúcej sa rozhodovaním agentov, sa používajú na tréning a hodnotenie staré ATARI hry. Tieto hry bežia v emulátore, čo značne spomaľuje tréning. Zároveň nemáme plný prístup k ich stavu, čo nám neumožnuje analyzovať agentov na celom stavovom priestore.

Táto práca sa zaoberá vytvorením generátora, ktorý je schopný vytvoriť úrovne podobné ATARI hrám, ako sú Pitfall! alebo Montezuma's Revenge, ktoré patria medzi najzložitejšie a zároveň najzaujímavejšie z pohľadu výskumu.

Generátor umožňuje používateľovi jednoducho vytvoriť nové úrovne nastavením malého počtu parametrov. Tiež je jeho súčasťou replikácia mechaník týchto hier, čo umožňuje tieto hry spustiť pre použitie s agentom umelej inteligencie.

Annotation

Slovak University of Technology Bratislava

FACULTY OF INFORMATICS AND INFORMATION TECHNOLOGIES

Degree course:	informatika
Autor:	Branislav Trstenský
Bachelor's Thesis:	Platform game generator
Supervisor:	Mgr. Matej Pecháč, PhD.

Január 2025

For reinforcement learning, in the field of artificial intelligence dealing with agent decision-making, old ATARI games are used for training and evaluation. These games run in an emulator, which significantly slows down training. At the same time, we do not have full access to their state, which does not allow us to analyze agents in the entire state space.

This thesis deals with the creation of a generator capable of generating levels similar to ATARI games such as Pitfall! or Montezuma's Revenge, which are among the most complex and at the same time most interesting from a research point of view.

The generator allows users to easily create new levels by setting a small number of parameters. It also includes a replica of the mechanics of these games, which allows these games to be run for use with an artificial intelligence agent.

Obsah

1 Úvod **1**

2 Stav existujúcich riešení **1**

3 Voľba spôsobu riešenia **2**

4 Opis riešenia **3**

4.1 Špecifikácia požiadaviek 4
4.2 Návrh 7
4.2.1 Generovanie úrovne 7
4.2.2 Herný svet a objekty 10
4.2.3 Generovanie objektov miestnosti 11
4.2.4 Fyzikálny systém 13
4.3 Implementácia 14
4.3.1 Herná slučka 14
4.3.2 Generovanie 15
4.3.3 Načítavanie herných prostriedkov 17
4.3.4 Editor súčastí miestnosti 18
4.4 Overenie riešenia 20

5 Zhodnotenie **20**

Literatúra **21**

1 Úvod

Pri spätnoväzbovom učení, v oblasti umelej inteligencie zaoberajúcej sa rozhodovaním agentov, sa používajú na tréning a jeho hodnotenie staré ATARI hry. Tieto hry bežia v emulátore, čo značne spomaľuje tréning. Zároveň nie je možný plný prístup k ich stavu, čoneumožňujeanalyzovať agentov na celom stavovom priestore.

Táto práca sa zaoberávytvorenímgenerátora, ktorý jeschopný vytvoriť úrovne podobné ATARI hrám, ako sú Pitfall! alebo Montezuma's Revenge, ktoré patria medzi najzložitejšie a zároveň najzaujímavejšie z pohľadu výskumu.

Generátor umožňujepoužívateľovi jednoducho vytvoriť nové úrovne nastavením malého počtu parametrov. Tiež je jeho súčasťou replikácia mechaník týchto hier, čo umožňuje tieto hry spustiť pre použitie s agentom umelej inteligencie.

Využitie aplikácie, ktorá je schopná prevádzky na moderných zariadeniach, odstraňuje značnú réžiu výkonu, v porovnaní s používaním emulátora. To umožňuje spustenie väčšieho množstva inštancií hry paralene na jednom zariadení.

Súčasťou tohto projektu je aj implementácia prostredia pre knižnicu Gymnasium, ktorá sa používa na sprostredkovanie komunikácie medzi UI agentmi a prostrediami. [3]

2 Stav existujúcich riešení

Strojové učenie agentov z vysokodimenzionálnch senzorických vstupov ako je zrak a reč je jedným z dlhodobých cieľov spätnoväzbového učenia. Trénovanie na situáciách z reálneho sveta, však vyžaduje veľké množstvo ručne označkovaných trénovacích dát. Algoritmy spätnoväzbového učenia však vyžadujú odmenu, ktorá je priamospojenás predošlými akciami agenta, takže sa vyžaduje, aby vstup pre učenie bol tvorený sekvenciou spojitých dát, ktoré sa dynamicky menia so správaním agenta.

Použitie hier umožňuje priamu reakciu na akcie agenta a konštantnú generáciu senzorických vstupov priamo z grafického výstupu hry. Hry tiež obsahujú indikátory úspechu agenta, t.j. hodnoty skóre, ktoré je možné použiť ako spätnú väzbu.

Hry pre hernú konzolu ATARI sa používajú, nakoľko ponúkajú množstvo komplexných úloh, ktoré boli navrhnuté tak, aby boli náročné pre ľudských hráčov. Pre interakciu agenta, bežiaceho na modernom hardvéri a hry navrhnutej pre hernú konzolu, je použitýemulátor. Ten umožňuje vloženie akcií vybraných agentom a následné prečítanie stavu hry, pre ich vyhodnotenie prostredníctvom skóre hry. Agent samotný nemá prístup k internému stavu hry, ale len k surovým obrazovým dátam. [5, 6]

Spustenie hry na emulátore však vytvára komplikácie v podobe značne zvýšenej potreby na výpočtové prostriedky oproti pôvodnej hre. Kvôli rozdielu architektúry medzimodernýmizariadeniami nie je možné priamo spustiť pôvodný strojový kód hry na zariadení, ale je potrebné simulovať virtuálnu konzolu, čo spôsobuje značnú réžiu, keďže pre každú strojovú inštrukciu hry je potrebné vykonať väčšie množstvo strojových inštrukcií na cieľovom zariadení. Existujú projekty, kde sú optimalizované výpočtové náklady pre emuláciu, napriklad použitím GPU prostredníctvom architektúry CUDA [1], ale aj v týchto prípadoch stále existuje réžia emulácie. Jedným z riešení tohto problému je re-implementácia hry pre moderné systémy alebo tvorba novej hry, ktorá je prispôsobená pre moderné systémy.

Pre strojové učenie je dôležité aby bol model generalizovaný, t.j. zabezpečenie toho, aby model dokázal vytvoriť dobré predpovede na doposiaľ neevidovaných dátach. V kontexte spätnoväzbového učenia, je to schopnosť reagovať na situácie, ktoré nenastali v procese učenia. Inak môže nastať pretrénovanie, čo pre spätnoväzbové učenie znamená zapamätanie si konkrétneho postupu krokov pre získanie odmeny, namiesto reakcií na aktuálny stav hry.

Napríklad bolo dokázané, že hoci UI agent na riadenie simulovaného auta po konkrétnej pretekárskej trati má dobré výsledky, výsledný model sa naučil riadiť iba túto konkrétnu trať. Postupným zahrnutím náročnejších úrovní do hodnotenia zdatnosti však možno vyvinúť model, ktorý dokáže dobre riadiť mnoho tratí, dokonca aj také, ktoré by sa model nebol schopný sa naučiť od začiatku. [9]

Použitie procedurálne generovaných prostredí umožňuje trénovanie modelu nakonštatnenových prostrediach, čo predchádza zapamätaniu si iba jedného prostredia a núti model naučiť sa reagovať na dopredu neznáme situácie. Pre strojové učenie je vhodné postupne zvyšovať obtiažnosť úrovní počas tréningu. Procedurálne generovanie umožňuje zmenu prostredia na základe viacerých parametrov, čo zabezpečí vhodné prispôsobenie obtiažnosti. [4]

3 Voľba spôsobu riešenia

Výsledkom tejto práce je generátor a herná aplikácia, ktorá replikuje herné mechaniky hry Montezuma's Revenge. Tvorba novej hry umožňuje spustenie kódu na moderných zariadeniach a priamy prístup k internému stavu hry. Hra Montezuma's Revenge bola vybraná ako cieľ replikácie, pre jej špecifickú štruktúru, t.j. každá úroveň je tvorená viacerými miestnosťami, čo vytvára vhodný cieľ pre procedurálnu generáciu jednotlivých úrovní.

Obr. 1: Snímka obrazovky hernej aplikácie

Hra obsahuje zberateľné odmeny vo forme drahokamov, nepriateľov rozmiestnených po miestnostiach, ako aj výzvu spočívajúcu v skokoch medzi platformami. Ich rozmiestnenie a počet je vhodným cieľom pre parametrizáciu.

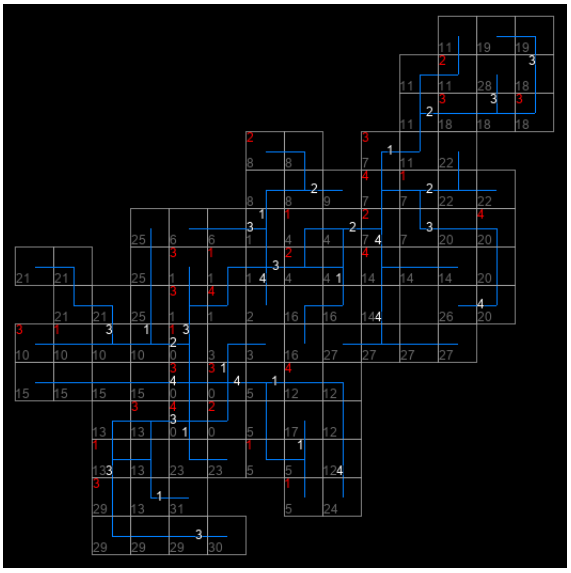
Tiež je tu výzva existujúca medzi miestnosťami, t.j. zbieranie kľúčov na odomknutie dverí pre ďalší postup cez úroveň.

Aplikácia je implementovaná v jazyku Python [8]pre jednoduchú inter-operáciu s knižnicou Gymnasium [3], ktorá sa používa na sprostredkovanie komunikácie medzi UI agentmi a prostrediami. Grafický výstup je generovaný cez knižnicu Pygame [7]. Aplikácia nemá žiadny zvukový výstup, keďže tento nie je potrebný pre tréning agenta.

4 Opis riešenia

Proces generovania úrovne spočíva vo vytvorení viacerých zón, ktoré sú tvorené miestnosťami. Zóny sú navzájom prepojené zamknutými dverami a v miestnostiach sú rozmiestnené kľúče potrebné pre ich otvorenie.

Rozmiestnenie miestností a ich prepojenia sú generované pomocou algoritmu na generovaniebludísk: prehľadávanie do hĺbky s náhodným spätným návratom. Na rozdiel od bludiskových generátorov tento proces pracuje na nekonečnej rovine, kde je obmedzený iba počet miestností (celkovo aj v jednotlivých zónach). Generovaná miestnosť môže tvoriť začiatok novej zóny, kde vstup do tejto miestnosti je zamknutý a zároveň je jediným vstupom do tejto zóny. Postupným tvorením zón od začiatočného bodu, sa definuje hierarchia, kde podradené zóny sú ďalej od začiatočného bodu. Je zabezpečené, aby každý zamknutý prechod do podradenej zóny, mal priradený kľúč umiestnený niekde v jednej z nadradených miestností.



Obr. 2: Vygenerovaná úroveň
Spojenie miestností je zobrazené modrou čiarou. Bielym textom je zobrazené, aký kľúč je potrebný na otvorenie prechodu a červeným textom je zobrazené, aký kľúč sa v miestnosti nachádza.

Obsah miestností je tvorený náhodným generátorom a kombinovaním dopredu vytvorených súčastí, podľa špecifikovaných parametrov. Každá súčasť obsahuje herné objekty, ktoré tvoria prostredie pre interakciu shráčskoupostavou, a zástupné objekty, ktoré sú rekurzívne nahradzované ďalšími súčasťami podľa daných parametrov.

Výsledkom tejto práce je aj program editora, ktorý umožňuje prostredníctvom grafického rozhrania definovať tieto súčasti, ich objekty a špecifikovať parametre pre ich generovanie.

Vygenerované úrovne sú hrateľné prostredníctvom hernej aplikácie. Aplikácia obsahuje grafický výstup, ako aj jednuduchú fyzikálnu simuláciu. Hra poskytuje štyri možné akcie: pohyb vľavo a vpravo, skok a lezenie hore a dole.

4.1 Špecifikácia požiadaviek

1. kompatibilita
   1.1. Program je kompatibilný s prostredím Gymnasium
   1.2. Program má výstup parametrov
      - čas
      - skóre
2. interakcia s používateľom
   2.1. Program prijíma používateľský vstup

- ísť hore
- ísť dole
- ísť doprava
- ísť doľava
- skok

2.1. Program má grafický výstup
- každý herný objekt musí byť reprezentovaný individuálnym grafickým prvkom

2.2. Program indikuje skóre

2.3. Program indikuje obsah inventáru

3. hráčska postava

3.1. Program umožňuje pohyb hráčkej postavy na zemi
- pohyb vpravo a vľavo

3.1. Program umožňuje skok zo zeme
- detekcia stavu na zemi alebo vo vzduchu

3.2. Program rieši kolíziu hráčkej postavy a geometrie miestnosti

3.3. Program umožňuje lezenie po rebríku
- pohyb hore a dole
- vstup a výstup zo stavu lezenia

3.4. Program umožňuje zbieranie objektov do inventára
- veľkosť inventára je obmedzená

3.5. Program umožňuje použitie predmetov z inventára
- interakcia je spustená automaticky pri kolízií s interaktívnym herným objektom
- pre každý herný objekt je aplikovateľný iba jeden predmet, nemôže vzniknúť nejasnosť
- použitý predmet je odobratý z inventára
- ak inventár obsahuje viacero aplikovateľných predmetov, je použitý iba jeden

3.6. Program reaguje na kontakt z objektami predstavujúcimi nebezpečenstvo
- pozícia hráča je nastavená na vstup do aktuálnej miestnosti

4. herný svet

4.1. Herný svet je rozdelený na jednotlivé miestnosti
- miestnosti ani objekty v nichneinteragujús inými miestnosťami
- v pamäti je načítaná iba aktuálna miestnosť, v ktorej je hráč prítomný

4.1. Stav objektov relevantných pre postup hry je zachovaný aj keď hráč opustí miestnosť
- stav dverí
- prítomnosť kľúčov
- prítomnosť drahokamov
- výsledok výberu zástupných objektov

4.2. Miestnosti sú umiestnené v mriežke
    - miestnosti v susedných bunkách sú nazývané susedné
    - susedné miestnosti môžu ale nemusia mať spojenie
    - miestnosť má 0 až 4 susedov
    - maximálne jeden sused pre každý zo štyroch smerov
4.3. Úroveň udáva zoznam miestností a ich pozíciu
4.4. Pre každú miestnosť je uložený stav spojenia pre všetky susedné miestnosti
    - práve jedno spojenie pre každého suseda
    - možné stavy:
        - zatvorenýpriechod
        - otvorený prechod
        - prechod na kľúč (daný druh kľúča)
    - ak neexistuje sused pre daný smer je stav nastavený ako zatvorený
4.5. Je umožnený prechod medzi miestnosťami
    - ak je spojenie definované ako:
        - zatvorené: prechod je blokovaný stenou
        - otvorené: prechod je voľný
        - na kľúč: sú postavené do cesty dvere
    - prechod je realizovaný herným objektom, ktorý načíta spojenú miestnosť pri kontakte s hráčom a prenesie hráča do nej
    - ak je prechodzatvorený, tento objekt nie je vytvorený pre daný smer
4.6. Každá miestnosť môže, ale nemusí mať kľúč
    - maximálne jeden kľúč pre miestnosť
4.7. Každá miestnosť má obsah
    - výber je obmedzený tak, aby obsah spĺňal požiadavky:
        - kompatibilita so stavmi vstupov miestnosti
        - prítomnosť kľúča, ak ho miestnosť má
    - ak miestnosť obsahuje zástupné objekty, tieto objekty sú rekurzívne nahradzované až kým nebudúnahradenévšetky

5. výzva
5.1. Miestnosti dávajú výzvu na zručnosti pohybu
    - skoky
    - nebezpečenstvo pádu do ohňa
5.1. Miestnosti dávajú výzvu na vyhýbanie sa nepriateľom
    - nebezpečenstvo kontaktu z nepriateľom
5.2. Celková úroveň dáva výzvu na orientáciu

    - nájdenie objektov nutných na pokrok
    - nájdenie cieľa úrovne
6. pokrok
    6.1. Zbieranie kľúčov na prechod cez zamknuté dvere
        - každá miestnosť môže, ale nemusí obsahovať kľúč
        - dvere aj kľúče majú danú farbu
        - na dvere je aplikovateľný iba kľúč rovnakej farby
        - nemôže vzniknúť situácia, kedy hráč nemá možnosť nájsť kľúč ku každým zamknutým dverám
    6.1. Zbieranie drahokamov na zvýšenie skóre
7. testovanie
    7.1. Program umožňujezobrazeniemapy celkovej úrovne
        - všetky miestnosti spolu so stavom prechodov
        - aktuálna pozícia hráča
        - poloha kľúčov
    7.1. Program umožňuje voľný prechod medzi miestnosťami
        - teleportácia hráča do každej zo susedných miestností
8. editor
    8.1. Editor umožňuje tvorbu a úpravu zoznamu možných miestností
    8.2. Editor je spustený ako separátny program
    8.3. Editor umožňuje vyskúšať vytvorenú miestnosť bez potreby spustenia hry
    8.4. Editor automaticky ukladá zmeny
    8.5. Editor umožňuje vrátiť zmeny
    8.6. Editor umožňuje vytvoriť inštanciu herných objektov z predom daného zoznamu
    8.7. Editor umožnuje presun a zmenu veľkosti objektov
    8.8. Editor umožňuje odstránenie objektov
    8.9. Editor umožňuje zmenu parametrov ovplyvňujúcu nahradenie zástupných objektov
    8.10. Miestnosti sú uložené vo formáte `json`

4.2 Návrh

4.2.1 Generovanie úrovne

Úroveň je tvorená množinou miestností umiestnených na mriežke. Počas generovania si program udržiava: zoznam existujúcich miestností, zoznam čakajúcich miestností a ukazovateľ na aktuálnu miestnosť a na aktuálnu zónu.

Každá miestnosť má definované pole spojení. Pole obsahuje štyri prvky, kde každý prvok reprezentuje jeden smer. Každý prvok je celé číslo s jednou z nasledujúcich hodnôt:

- $-1 \Rightarrow$ žiadne spojenie
- $0 \Rightarrow$ spojenie, ktoré nie je zamknuté
- $\geq 1 \Rightarrow$ typ kľúča, ktorým je spojenie zamknuté

Typy kľúčov sú rozlišované na základe farby, kde kde ľubovoľný kľúč istej farby otvára dvere prislúchajúce tejto farbe. Všetky možné kľúče sú definované jednoduchým poľom farieb, takže je možné ich počet zmeniť alebo prispôsobiť odtiene farby, ak má agent problém ich rozlíšiť.

Taktiež každá miestnosť obsahuje jej pozíciu na mriežke úrovne a číslo zóny, v ktorej sa nachádza. Pri generovaní sa pre každú miestnosť udržiava pole smerov, ktoré ešte neboli preskúmané.

Každá zóna je označená poradovým číslom. Pre každú zónu je udržiavané pole všetkých miestností, ktoré obsahuje.

Prvým krokom generátora je vytvoriť rozloženie miestností. Program najprv vytvorí miestnosť na pozícií $[0, 0]$ a vloží ju do zoznamu existujúcich a čakajúcich miestností. Následne podľa parametrov generuje nové miestnosti až pokiaľ nedosiahne rozloženie spĺňajúce požiadavky definované parametrami.

Parametre generátora sú:

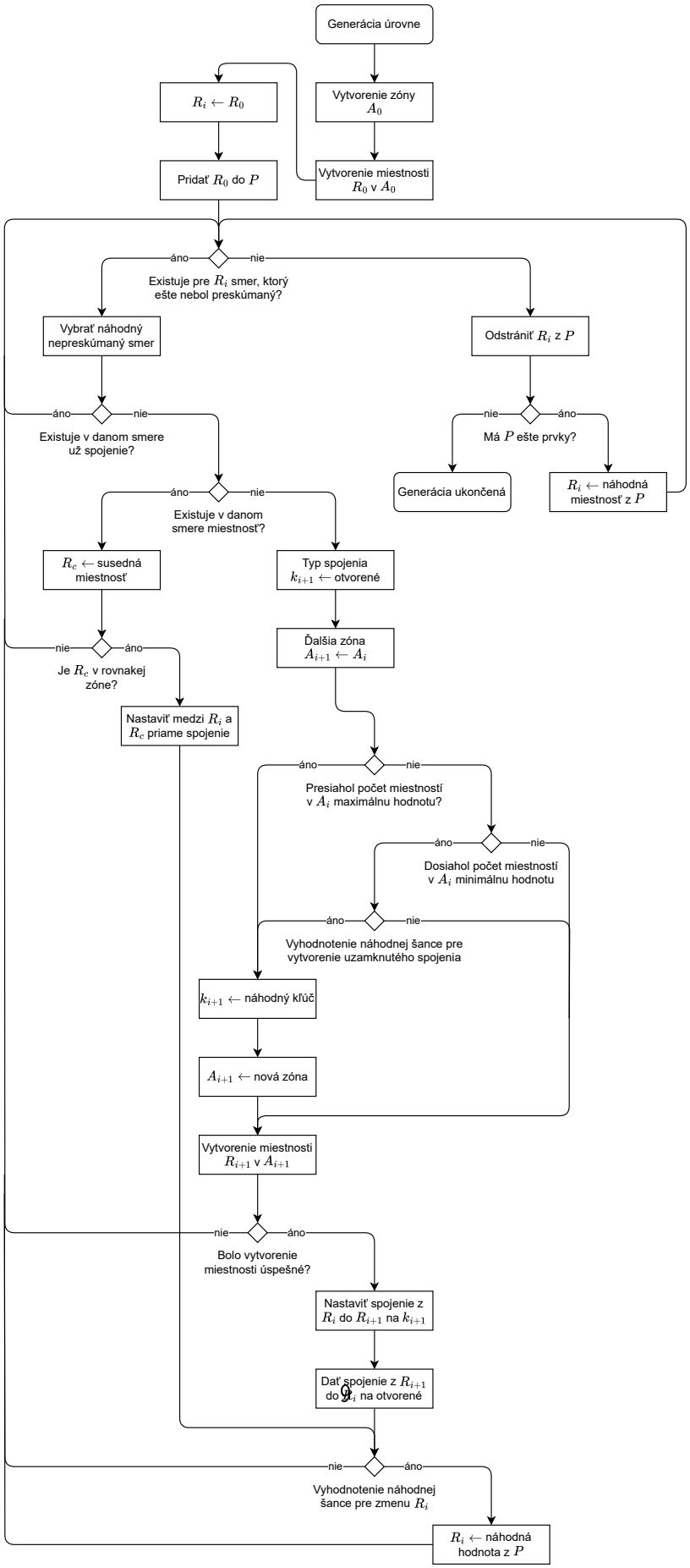
- Maximálny počet miestností
- Maximálna šírka úrovne
- Maximálna výška úrovne
- Minimálny počet miestností pre zónu
- Maximálny počet miestností pre zónu
- Veľkosť štartovacej zóny
- Šanca vytvoriť uzamknuté spojenie
- Šanca vytvoriť spätný návrat

Stav generátora je nasledovný:

- $A_i \Rightarrow$ aktuálna zóna
- $R_i \Rightarrow$ aktuálna miestnosť
- $A \Rightarrow$ zoznam všetkých zón
- $R \Rightarrow$ zoznam všetkých miestností
- $P \Rightarrow$ zoznam čakajúcich miestností
- $x_{\text{min}}, x_{\text{max}}, y_{\text{min}}, y_{\text{max}} \Rightarrow$ minimálne a maximálne dosiahnuté hodnoty pozície miestností
- $K \Rightarrow$ zoznam potrebných kľúčov

Priebeh generovania rozloženia miestností je ilustrovaný v diagrame 9. Generovanie sa odvoláva na nasledovné procedúry:

- **Procedúra** `pridať_miestnosť`



Obr. 3: Generovanie úrovne

- **Vstup:** pozícia miestnosti L, číslo zóny a
- **Výstup:** vytvorená miestnosť

1. **Ak** je pozícia miestnosti mimo rozsahu definovaného $x_{\text{min}}, x_{\text{max}}, y_{\text{min}}, y_{\text{max}}$
   1.1. Je vypočítaný nový rozsah
   1.2. **Ak** je definovaná maximálna šírka alebo výška a nový rozsah ju prekračuje
      1.2.1. Funkcia je prerušená
   1.3. Rozsah je aktualizovaný na nový rozsah
2. Vytvorená miestnosť R_{i+1} z pozíciou L a číslom zóny a
3. Miestnosť R_{i+1} pridaná do zoznamu miestností R
4. Miestnosť R_{i+1} pridaná do zoznamu miestností v zóne A_a
5. **Výstup** R_{i+1}

- **Procedúra** `pridať_zónu`
  - **Vstup:** nadradená zóna A_i, ak existuje
  - **Výstup:** vytvorená zóna

  1. $i \leftarrow$ poradové číslo zóny
  2. Vytvorená zóna A_{i+1}, s poradovým číslom i a nadradenou zónou A_i
  3. Zóna A_{i+1} pridaná do zoznamu zón A
  4. **Výstup** A_{i+1}

Po vytvorení rozloženia miestností, program rozmiestni kľúče potrebné k na otvorenie vytvorených uzamknutých spojení. Program iteruje cez všetky zamknuté prechody v poradí od tých v zónach najbližších k začiatku úrovne až ku najvzdialenejším. Toto je vykonané pomocou porovnávania poradového čísla zóny. Program potom nájde miestnosť, kde je možné vložiť kľúč tak, aby nevznikla situácia, že hráč môže daný kľúč získať iba po otvorení prechodu, ktorý odomyká.

4.2.2 Herný svet a objekty

Herné objekty sú inštanciami tried odvodených z jednej nadtriedy. Prepísaním metód a modifikáciou polí nadtriedy, herné objekty majú definovanú svoju interakciu s fyzikálnym systémom, ich grafickú reprezentáciu a ich správanie pri generovaní miestností.

Aby boli herné objekty funkčné, musia byť pridané do sveta. Svet je trieda, ktorá riadi životnosť herných objektov, ich vykresľovanie a interakciu medzi nimi. V tom istom čase môže existovať viacero svetov, ale iba jeden je aktívny - čo znamená, že v ňom prebieha čas, je vykresľovaný na obrazovku a hráč s ním môže interagovať.

Aktívny svet je uložený v objekte triedy vesmíru. Trieda vesmíru taktiež umožňuje herným objektom prístup k inštancii tried, prostredníctvom ktorých zakresľujú svoju grafickú reprezentáciu na obrazovku a prijímajú vstup odpoužívateľa.

Všetky herné objekty obsahujú nasledovné polia:

- Pozícia ⇒ dvojrozmerný vektor, ktorý určuje polohu objektu vo svete
- Veľkosť ⇒ dvojrozmerný vektor, ktorý určuje veľkosť objektu
- Referencia na objekt sveta, v ktorom sa objekt nachádza
- Referencia na objekt vesmíru, v ktorom sa objekt nachádza
- Vrstva ⇒ enumerácia, ktorá udáva prioritu pre vykresľovanie na obrazovku
- Kolízne príznaky ⇒ definujú spôsob interakcie s objektom vo fyzikálnom systéme

Herný objekt môže patriť do jednej z nasledovných vrstiev, pričom objekt v neskoršej vrstve je vždy vykreslený nad objektom v predchádzajúcej vrstve.

- Pozadie
- Normálny
- Grafické rozhranie

Herný objekt môže mať nasledovné kolízne príznaky:

- Trigger ⇒ umožňuje reagovať na prekryt s iným objektom
- Statický ⇒ fyzikálny systém predchádza kolízii s daným objektom

Triedy definujúce herné objekty môžu reagovať na rôzne udalosti, buď v hernom svete alebo v ich životnom cykle prostredníctvom prepísania metód. Udalosti, na ktoré triedy môžu reagovať sú nasledovné:

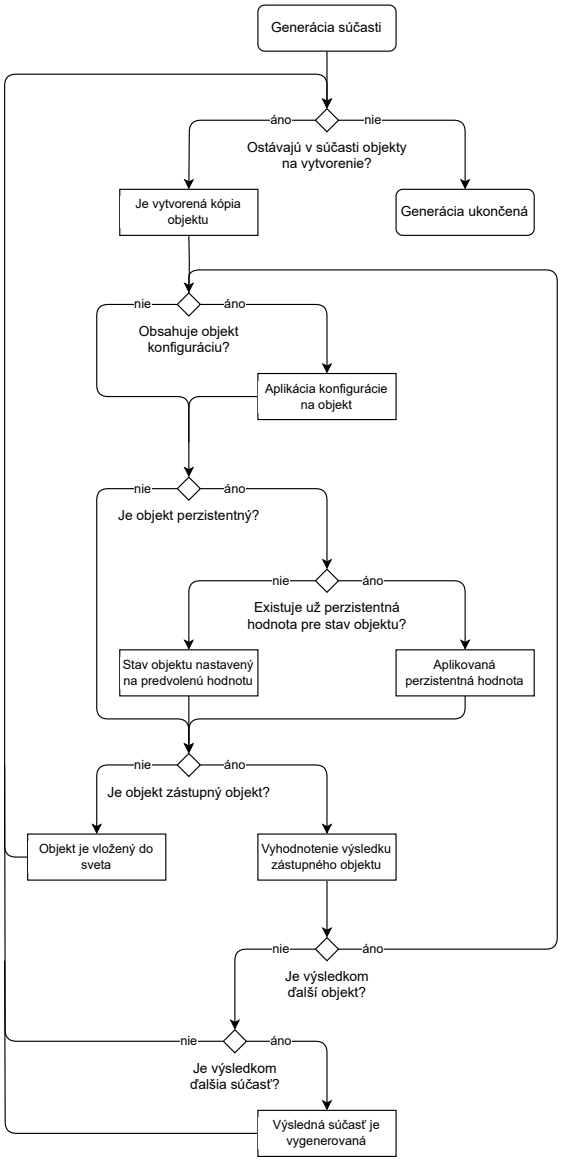
- Vykreslenie
- Posun času
- Vytvorenie objektu
- Pridanie objektu do sveta
- Odstránenie objektu zo sveta
- Prekryt s objektom typu trigger

4.2.3 Generovanie objektov miestnosti

Pre šetrenie výpočtových prostriedkov, sú objekty miestností načítané vždy iba pre miestnosť, v ktorej sa hráč aktuálne nachádza. Všetky objekty v miestnosti sú vygenerované práve v bode, kedy hráč do miestnosti vstúpi a zároveň sú všetky objekty vymazané hneď ako z miestnosti odíde.

Každá miestnosť je reprezentovaná separátnym svetom. Tento svet je vytvorený pri vstupe používateľa a je v ňom spustený proces generácie miestnosti. Obsah miestností je tvorený náhodným generátorom a kombinovaním dopredu vytvorených súčastí, podľa špecifikovaných parametrov. Tieto súčasti sú tvorené zoznamom herných a zástupných objektov.

Zástupný objekt je podtrieda herného objektu, ktorá obsahuje konfiguráciu, podľa ktorej je vybraný herný objekt alebo ďalšia súčasť, ktorej objekty sú rekurzívne spracované. Konfigurácie pre tento výber obsahuje podmienky, ktoré používateľ vyberie tak, aby sa generácia miestnosti vyvíjala podľa parametrov a náhodného generátora. Proces generovania objektov je ilustrovaný v diagrame 4.



Obr. 4: Generácia objektov miestnosti

Parametre môžu byť globálne - tie ktoré určujú obtiažnosť celej úrovne, alebo lokálne parametre, t.j. typy spojení, ktoré miestnosť má.

Globálne parametre sú nasledovné:

- Obtiažnosť nepriateľov
- Obtiažnosť skokov
- Počet odmien

Jedinou výnimkou je koreňová súčasť, ktorá je vybraná pri generácií celej úrovne. Táto má v sebe definované, ktoré spojenia podporuje a či má možnosť obsahovať v sebe kľúč. Pre každú miestnosť v úrovni je náhodne vybraná koreňová súčasť z množiny tých, ktoré podporujú požiadavky miestnosti. Toto umožňuje dopredu zistiť, či existuje miestnosť, ktorej požiadavky nie je možné naplniť a generovanie prerušiť.

Keďže sú všetky objekty miestnosti odstránené pri odchode hráča, dôsledky jeho interakcie s objektami ako je napríklad zbieranie odmien alebo odomykanie prechodov, by boli stratené. Preto každá miestnosť obsahuje pole perzistentných hodnôt, do ktorej objekty nastavené ako perzistentné, ukladajú svoj stav. Výsledok vyhodnotenia zástupných objektov je tiež uložený, aby sa tieto výpočty nemuseli opakovať.

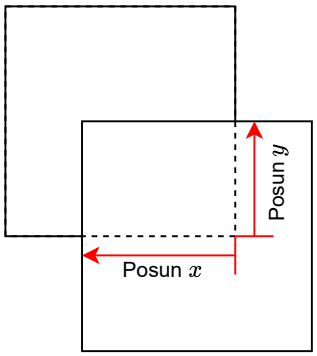
4.2.4 Fyzikálny systém

Pre umožnenie pohybu hráča a jeho interakciu s objektami v miestnosti, aplikácia implementuje jednoduchý fyzikálny systém. Je zavedená iba jednoduchá detekcia kolízií a statické riešenie kolízií.

Pre detekciu kolízií sú objekty redukované na jednoduchý obdĺžnik, odvodený od ich pozície a veľkosti. Objekty, ktoré majú nastavené kolízne príznaky trigger alebo statické, sú uložené do polí. Keď chce objekt zistiť či je v kolízii s jedným z týchto objektov, je jeho obdĺžnik porovnaný s obdĺžnikmi týchto objektov.

V prípade detekcie objektov typu trigger, je pri detekcii prekrytu spustená na obidvoch kolíznych objektoch metóda obsluhujúca túto udalosť.

V prípade detekcie statických objektov je kolízia vyriešená posunom kolízneho objektu - konkrétne toho, ktorý nie je statický. Porovnaním rozdielu hrán obdĺžnikov reprezentujúcich obidva objekty v osi x a y je vypočítaný najmenší možný posun, ktorý je následne aplikovaný na kolízny objekt.



Obr. 5: Riešenie kolízií

Kolízie sú spracované len pre objekty, ktoré si to explicitne vyžiadajú. Konkrétne sa tento systém používa v týchto prípadoch:

- Kolízia hráča so stenami
- Kolízia nepriateľov so stenami
- Interakcia hráča s interaktívnymi objektmi
- Interakcia hráča s nepriateľmi

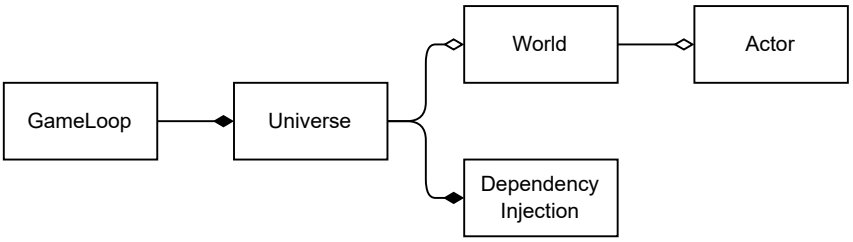
4.3 Implementácia

Aplikácia je implementovaná v jazyku Python [8]pre jednoduchú inter-operáciu z knižnicou Gymnasium [3], ktorá sa používa na sprostredkovanie komunikácie medzi UI agentmi a prostrediami. Grafický výstup je generovaný cez knižnicu Pygame [7].

Prístup k grafickému výstupu a používateľskému vstupu je abstrahovaný tak, aby sa kód hry nemusel modifikovať medzi interaktívnym riadením človekom alebo riadením agentom umelej inteligencie.

Ako bolo spomenuté v sekcií 4.2.2, hra je stavaná z herných objektov, ktoré sú umiestnené do sveta, ktorý je následne umiestnený do vesmíru. Táto architektúra je implementovaná následovnými triedami:

- `Actor` ⇒ Nadtrieda všetkých herných objektov
- `World` ⇒ Reprezentuje herný svet
- `Universe` ⇒ Kontajner pre objekty, s ktorými herné objekty interagujú
- `DependencyInjection` ⇒ Umožňujú herným objektom získať referencie na podporené objekty
- `GameLoop` ⇒ Umožňuje interakciu používateľa s hrou

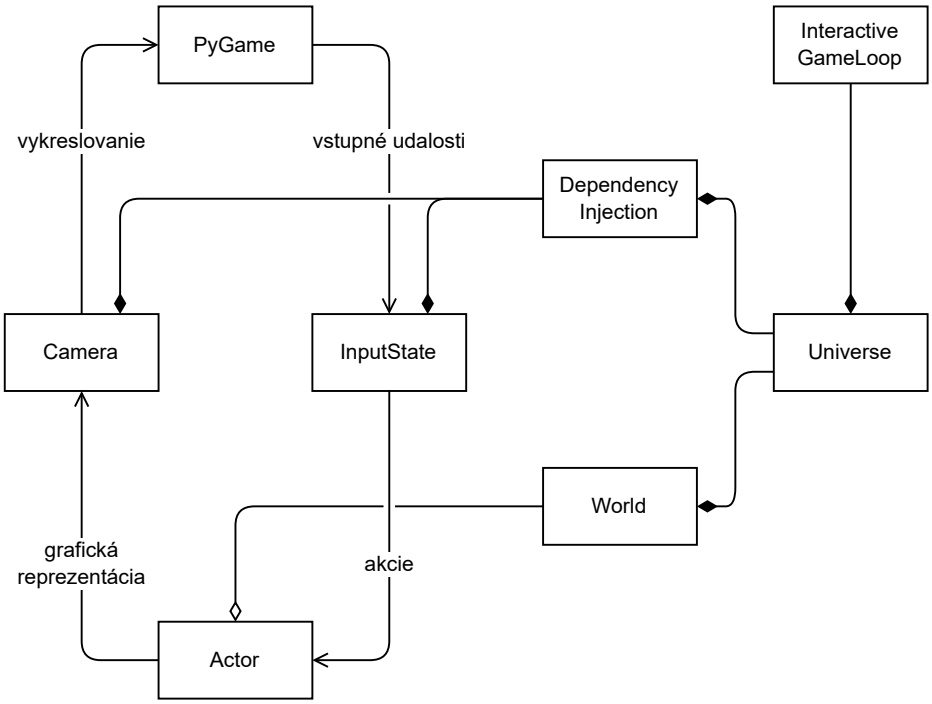


Obr. 6: Štruktúra hry

4.3.1 Herná slučka

Interakciu herných objektov a hráča sprostredkúvajú podporné objekty, ktoré sú vytvorené triedou `GameLoop` a registrované v registri `DependencyInjection`. Keď chcú herné objekty tvoriť grafický výstup alebo konzumovať vstup od používateľa, vyžiadajú si inštanciu týchto objektov z registra. Podporné objekty pre tento účel sú nasledovné:

- `Camera` ⇒ Generuje grafickú reprezentáciu herných objektov
- `InputState` ⇒ Poskytuje prístup k vstupným akciam hráča



Obr. 7: Funkcia podporných objektov pre interakciu s používateľom

Pri použití hry človekom sa používa podtrieda `InteractiveGameLoop`, ktorá vytvorí herné okno a zbiera vstup z klávesnice. Pri použití hry agentom umelej inteligencie sa modifikuje stav v `InputState` podľa rozhodnutí agenta a grafický výstup hry sa uloží do poľa pixelov, ktorý tvorí vstup pre agenta.

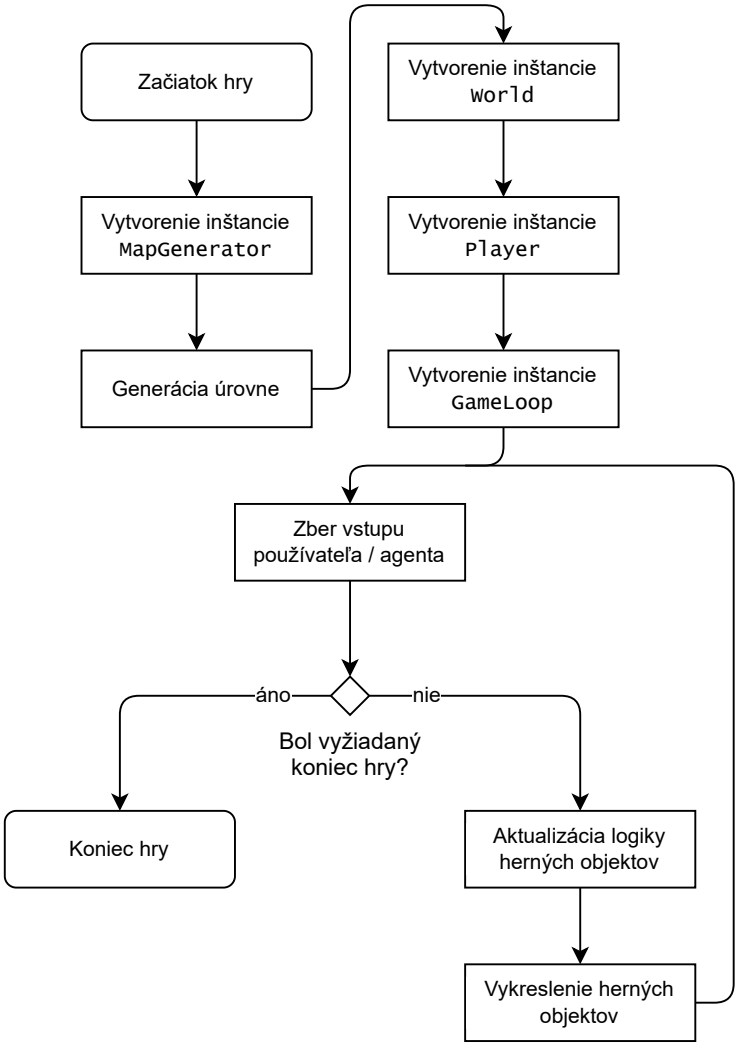
Pred spustením hry je vygenerovaná úroveň prostredníctvom triedy `MapGenerator`. Následne je vytvorený `World` pre prvú miestnosť a je do nej umiestnený hráč. Nakoniec je vytvorená inštancia `GameLoop` a herná slučka sa vykonáva, až kým nie je zatvorené herné okno človekom, resp. kým nie je ukončené trénovanie agenta. Proces hernej slučky je ilustrovaný v diagrame 8.

4.3.2 Generovanie

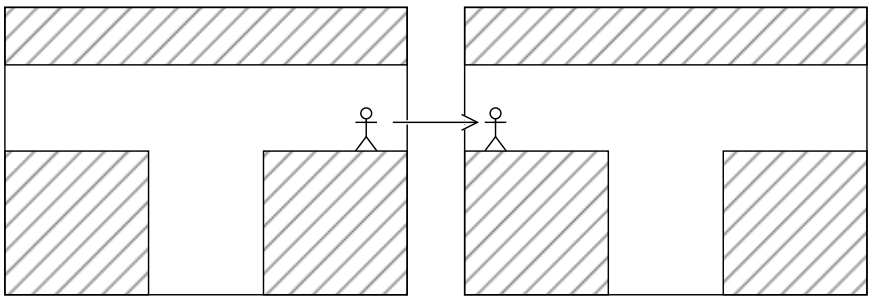
Generovanie mapy má na starosti trieda `MapGenerator`, ktorá vytvorí viacero objektov `RoomInfo`, ktoré obsahujú informácie o miestnostiach v úrovni a o spojeniach medzi nimi.

Súčasti miestností sú definované objektami `RoomPrefab`. Každý objekt `RoomInfo` má referenciu na `RoomPrefab`, ktorý tvorí prvú koreňovú súčasť pre generáciu objektov v miestnosti.

Generovanie miestnosti má na starosti objekt triedy `RoomController`, ktorý vytvorí objekt `World` a spustí generovanie objektov miestnosti. Následne umiesti hernú postavu na správne miesto - a to tak, aby stál v logickom spojení s miestnosťou, z ktorej do novej miestnosti prišiel.



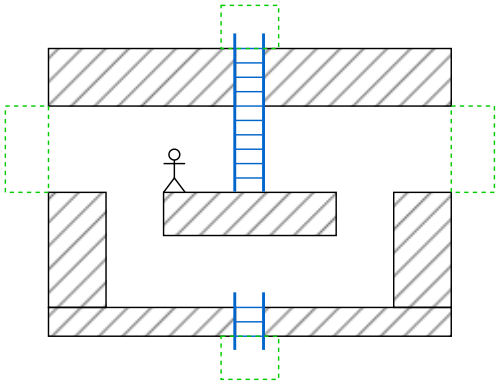
Obr. 8: Herná slučka



Obr. 9: Umiestnenie hernej postavy pri prechode medzi miestnosťami

Pri generovaní miestnosti je vytvorení objekt `RoomInstantiationContext`, ktorý obsahuje všetky parametre pre jej generovanie. Načítanie objektov v súčasti, má na starosti trieda `LevelDeserializer`, ktorá pri každom načítaní objektu, spustí spätné volanie do `RoomInstantiationContext` pre spracovanie objektu podľa diagramu 4.

Na stranách miestnosti sú v prípade prítomnosti spojenia s inou miestnosťou vytvorené interaktívne objekty triedy `RoomTrigger`, ktoré pri kontakte s hráčom, ho presunú do spojenej miestnosti.

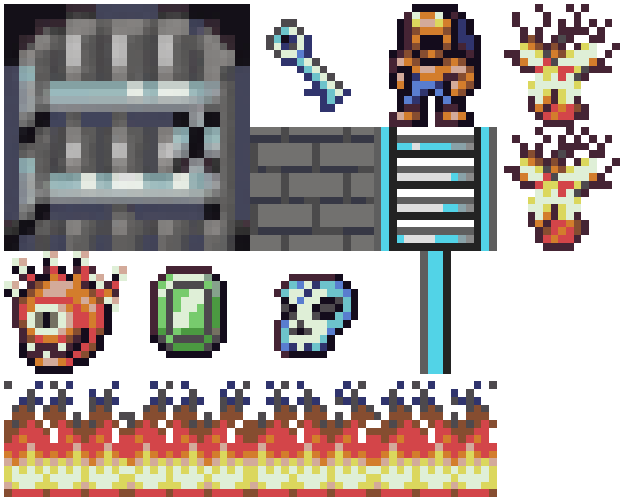


Obr. 10: Herné objekty, ktoré presunú hráča do spojenej miestnosti

Pre diagnostiku generovania pri vývoji alebo pre možnosť získania viac informácií pre vyhodnocovanie tréningu agenta, je vytvorený podporný objekt `MapView`, ktorý pri stlačení tlačidla `M` zobrazí mapu úrovne a umožňuje stlačením kláves `I`, `J`, `K` a `L` premiestňovať hráčsku postavu medzi miestnosťami.

4.3.3 Načítavanie herných prostriedkov

Hra definuje podporný objekt `ResourceProvider`, ktorý načíta grafickú reprezentáciu herných objektov a písma pre zobrazovanie textu v hre. Tieto dáta sú načítané prostredníctvom funkcií obsiahnutých v PyGame. Všetky textúry sú uložené v jednom súbore vo forme atlasu. Referenciu na tento objekt môžu herné objekty získať prostredníctvom `DependencyInjection`.



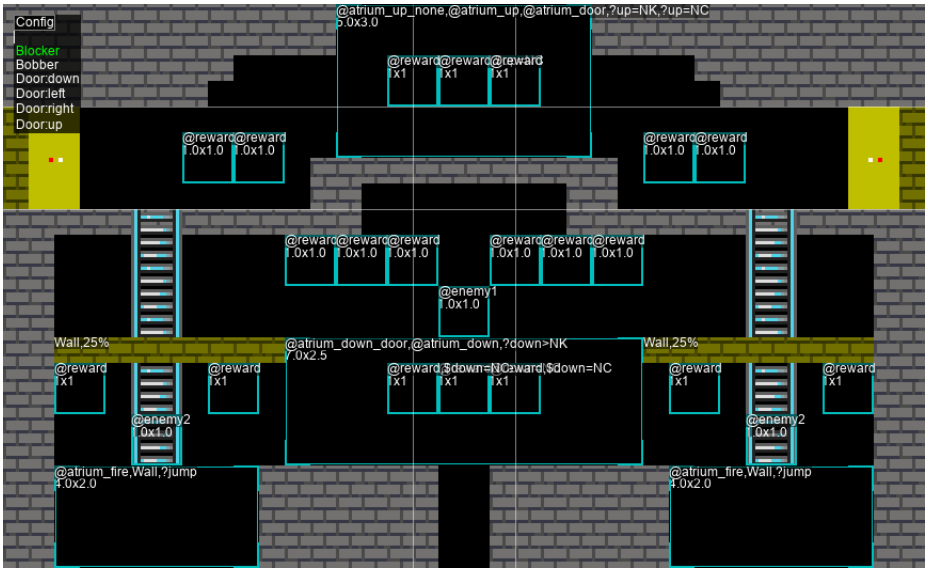
Obr. 11: Grafické prostriedky uložené v atlase

Všetky možné súčasti miestnosti sú triedou `RoomPrefabRegistry` načítané z `json` súborov. Každá miestnosť obsahuje zoznam herných objektov, ktoré sa majú vytvoriť pri generovaní. V súbore sú tieto herné objekty reprezentované svojím menom. Aby bolo možné z ich mena vytvoriť inštanciu objektu, sú všetky objekty, ktoré je dovolené používateľom umiestniť, uložené v triede `ActorRegistry`.

4.3.4 Editor súčastí miestnosti

Na tvorbu jednotlivých súčastí miestnosti je implementovaná aplikácia editora, ktorá prostredníctvom grafického rozhrania umožňuje definovať herné objekty súčasti a taktiež vytvoriť a konfigurovať zástupné objekty.

Editor sa spustí prostredníctvom konzolového rozhrania, kde sa ako argument k príkazu špecifikuje `json` súbor, v ktorom je uložená súčasť, ktorú sa používateľ rozhodol editovať (ak špecifikovaný súbor ešte neexistuje, je vytvorený). Editor zdieľa rovnakú architektúru ako samotná hra s tým rozdielom, že namiesto objektu hráča je vytvorený objekt triedy `LevelEditor`. Tento objekt prevádzkuje samotné editovanie ako aj ukladanie zmien do súboru.



Obr. 12: Aplikácia editora

Používateľ môže vytvoriť nové objekty ich vyhľadaním prostredníctvom mena a kliknutím pravého tlačidla myši na miesto, kde má byť objekt vložený. Je možné kliknutím ľavého tlačidla vybrať objekt a ťahaním myšou ho presunúť na novú pozíciu. Taktiež je možné zmeniť veľkosť zvoleného objektu pomocou ťahania držadiel na stranách vybraného objektu. Stlačením klávesu `DELETE`, je vybraný objekt vymazaný, stlačením kombinácie `CTRL+D` je zvolený objekt duplikovaný.

Po vybratí zástupného objektu je možné jeho konfiguráciu upraviť písaním textu. Konfigurácia je tvorená jednoduchým zásobníkovým programovacím jazykom, prostredníctvom ktorého je možné nastaviť, ktorým herným objektom alebo súčasťou bude zástupný objekt nahradený. Príkazy sú oddelené čiarkou. Neplatná konfigurácia je indikovaná zmenou farby textu na červenú. Konfigurácia obsahuje nasledovné príkazy:

- % + percento ⇒ je vybraná hodnota zo zásobníka. Podľa náhodnej šance danej percentom, je buď hodnota vložená naspäť alebo nahradená prázdnou hodnotou
- % + percento + ? ⇒ sú vybrané dve hodnoty zo zásobníka. Podľa náhodnej šance danej percentom, je buď vložená naspäť prvá hodnota alebo druhá
- $ + podmienka ⇒ je vybraná hodnota zo zásobníka. Ak je daná podmienka platná, je hodnota vložená naspäť inak je nahradená prázdnou hodnotou
- ? + podmienka ⇒ sú vybrané dve hodnoty zo zásobníka. Ak je daná podmienka platná, je prvá hodnota vložená naspäť inak je vložená druhá hodnota
- @ + názov skupiny súčasti ⇒ vloží skupinu súčasti na zásobník
- názov herného objektu ⇒ vloží herný objekt na zásobník

Stlačením tlačidla "Config" sa otvorí menu, prostredníctvom ktorého je možné nastaviť, ktoré požiadavky súčasť spĺňa, pre výber ako koreňovú miestnosť, alebo je možné vybrať parametre generovania, prípadne pridať súčasť do skupín pre výber do zástupných objektov.

Stlačením klávesu `ENTER` je možné vyskúšať editovanú súčasť. Za funkcionalitu je zodpovedná trieda `TestPlayController`, ktorá vytvorí dočasnú kópiu editovanej súčasti a na pozícií myši je vytvorený objekt hráča. Stlačením tlačidla "Config" sa otvorí menu, prostredníctvom ktorého je možné aktivovať generovanie miestnosti s aktuálne editovanou súčasťou ako koreňovou súčasťou a nastaviť parametre, podľa ktorých generovanie prebieha. Stlačením klávesu `TAB` je testovací svet znovu vygenerovaný alebo opätovným stlačením klávesu `ENTER` je možné vrátiť sa naspäť do editora.

Všetky zmeny sú automaticky ukladané. Editor implementuje aj históriu zmien, stlačením kombinácie `CTRL+Z` je možné vrátiť späť poslednú akciu alebo `CTRL+SHIFT+Z` je možné vrátenú akciu znovu aplikovať.

4.4 Overenie riešenia

Keďže projekt ešte nie je dokončený, nie je možné jednoznačne hodnotiť správnosť riešenia. Projekt už spĺňa požiadavky pre interakciu hráča s herným prostredím a pre generovanie úrovne a miestností podľa stanovených parametrov, čo bolo overené testovaním hry človekom.

5 Zhodnotenie

Cieľom práce bolo vytvorenie generátora, ktorý umožňujepoužívateľovi jednoducho vytvoriť herné prostredie pre trénovanie umelej inteligencie. Generátor tvorí neustále nové úrovne nastavením iba malého počtu parametrov. Súčasťou práce je aj replikácia mechaník hry Montezuma's Revenge, čo umožňuje trénovanie agenta umelej inteligencie bez potreby emulácie starších zariadení.

Výsledkom práce je generátor prostredia a herná aplikácia, ktorá umožní hranie hry človekom alebo agentom UI.

Aplikácia je implementovaná v jazyku Python [8]. Jej súčasťou je aj implementácia prostredia pre knižnicu Gymnasium [3]. Grafický výstup je generovaný cez knižnicu Pygame [7]a grafická reprezentácia objektov používa obrázky z balíčka DawnLike [2]distribuovaného pod licenciou CC-BY 4.0.

Generátor prostredia, ktorý dokáže vytvárať neustále nové úrovne, môže pomôcť k robustnejšiemu tréningu agentov umelej inteligencie pri spätnoväzbovom učení. Herná aplikácia umožňuje tréning vykonávať bez potreby použitia emulátora, čo pomôže spustiť viacero inštancií trénovania paralelne.

Literatúra

[1] DALTON, S.; FROSIO, I. GARLAND, M.: Accelerating Reinforcement Learning through GPU Atari Emulation. 2020, 1907.08467. Dostupné z: https://arxiv.org/abs/1907.08467

[2] DragonDePlatino DawnBringer: DawnLike - 16x16 Universal Rogue-like tileset v1.81 | OpenGameArt.org. Január 2014. Dostupné z: https://opengameart.org/content/dawnlike-16x16-universal-rogue-like-tileset-v181

[3] Farama: Basic Usage - Gymnasium Documentation. December 2024. Dostupné z: https://gymnasium.farama.org/introduction/basic_usage/

[4] JUSTESEN, N.; TORRADO, R. R.; BONTRAGER, P. et al.: Illuminating Generalization in Deep Reinforcement Learning through Procedural Level Generation. 2018, 1806.10729. Dostupné z: https://arxiv.org/abs/1806.10729

[5] KAISER, L.; BABAEIZADEH, M.; MILOS, P. et al.: Model-Based Reinforcement Learning for Atari. 2024, 1903.00374. Dostupné z: https://arxiv.org/abs/1903.00374

[6] MNIH, V.: Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013. Dostupné z: https://people.engr.tamu.edu/guni/csce642/files/dqn.pdf

[7] Pygame contributors: Pygame Front Page — pygame v2.6.0 documentation. August 2024. Dostupné z: https://www.pygame.org/docs/

[8] Python Software Foundation: The Python Tutorial — Python 3.13.1 documentation. December 2024. Dostupné z: https://docs.python.org/3/tutorial/index.html

[9] TOGELIUS, J. LUCAS, S.: Evolving robust and specialized car racing skills. Január 2006, s. 1187 – 1194, doi:10.1109/CEC.2006.1688444.